# Cart System - Technical Documentation

**Version:** 2.1.6
**Package:** @ravishranjan/cart
**License:** GPL v2.0
**Author:** Ravish Ranjan

# Table of Contents

1. Overview
2. Architecture
3. Core API
4. React Integration
5. Storage Management
6. Type Definitions
7. Configuration
8. Build & Distribution

---

# Overview

Cart is a lightweight, framework-agnostic shopping cart system designed for frontend applications. It provides:

- **Native JavaScript API** for vanilla JavaScript, Vue, Svelte, and other frameworks
- **React-specific bindings** via hooks and context API
- **Persistent storage** using browser localStorage or sessionStorage
- **Multi-format distribution** (CommonJS, ES modules, and global UMD)
- **TypeScript support** with full type definitions

## Key Features

| Feature | Details |
|---|---|
| **Storage Flexibility** | localStorage or sessionStorage support |
| **Framework Agnostic** | Works with any JavaScript framework |
| **Category Support** | Filter items by category |
| **Extensible** | Custom properties on cart items |
| **Browser Native** | No external dependencies |
| **Type-Safe** | Full TypeScript support |

# Architecture

## Module Structure

```
src/
├── core/
│   ├── cart.ts              # Core Cart class
│   ├── storage.ts           # Storage abstraction layer
│   └── cart.global.ts       # Global/UMD entry point
├── react/
│   ├── CartContext.tsx      # React context definition & provider
│   ├── useCart.tsx          # React hook for consuming context
│   └── index.ts             # React exports
└── index.ts                 # Main entry point
```

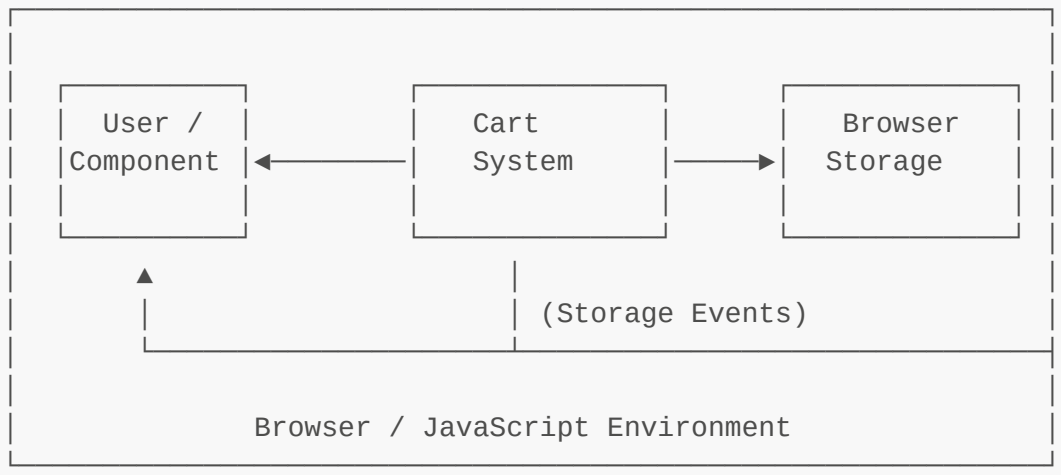
## Distribution Outputs

| Format | Location | Purpose |
|---|---|---|
| **CommonJS** | `dist/index.js` | Node.js and bundlers with CJS support |
| **ES Module** | `dist/index.mjs` | Modern bundlers and native imports |
| **Global/UMD** | `dist/cart.min.js` | Browser `<script>` tag usage |
| **TypeScript** | `dist/types.d.ts`, `dist/index.d.ts` | IDE autocomplete and type checking |

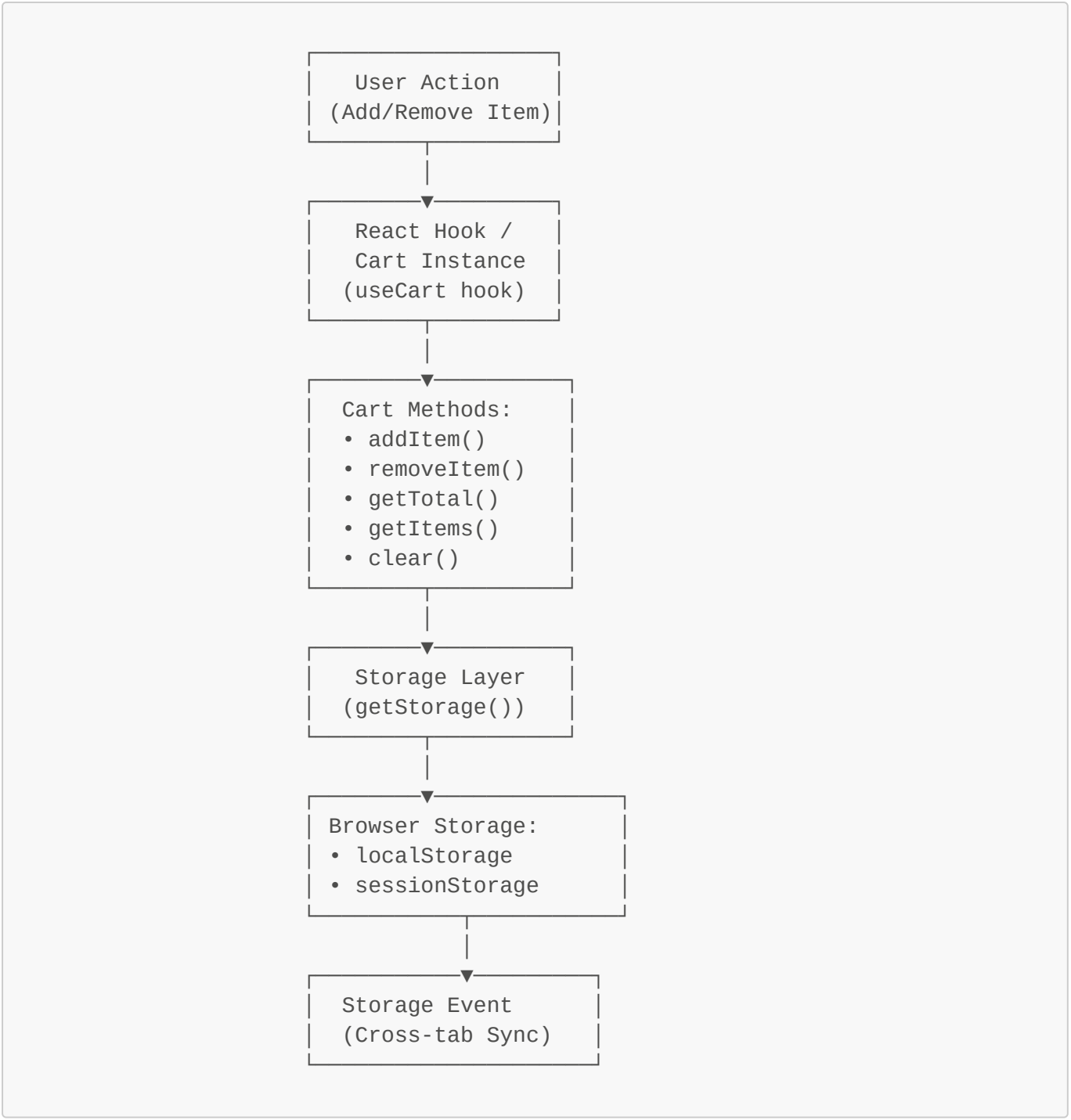
---

## Data Flow Diagram (DFD)

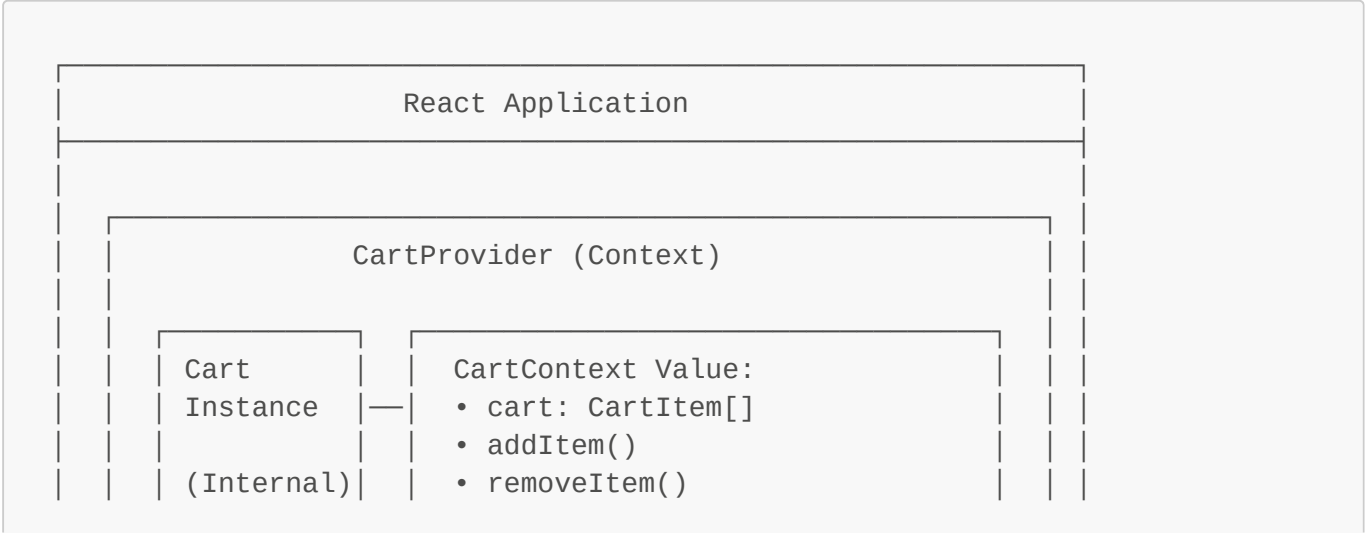
### Level 0 - System Context

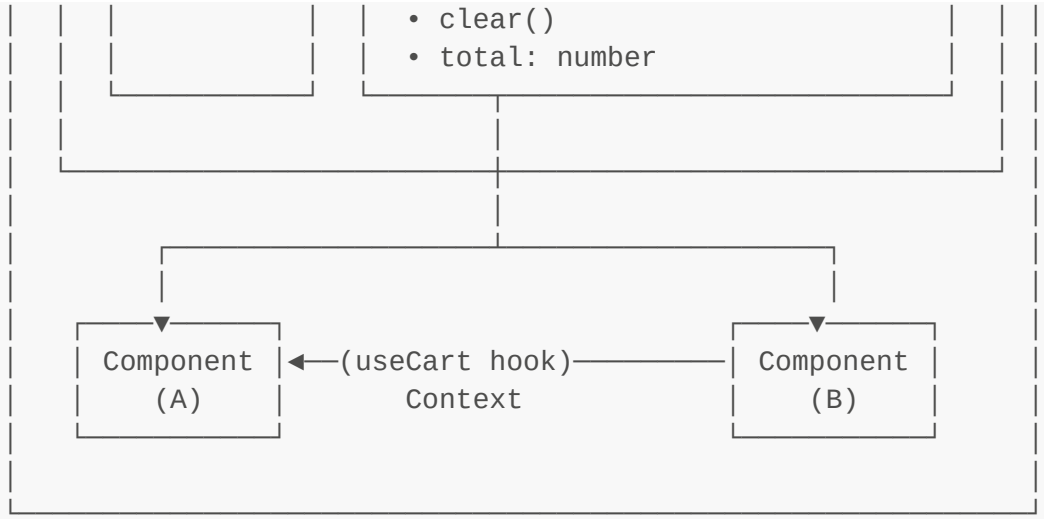



### Level 1 - Core Processes

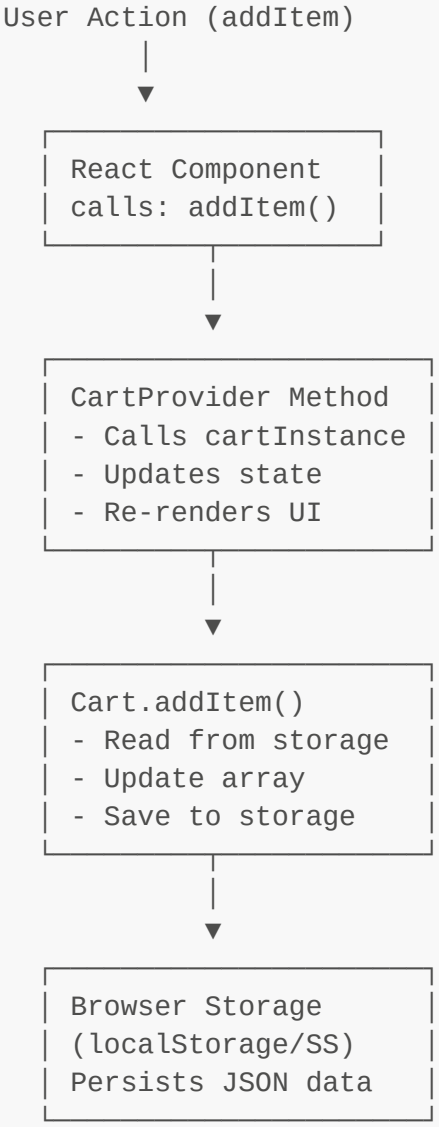



**Level 2 - React Integration Flow**






**Data Flow: Adding an Item**




**Data Format in Storage**

```
localStorage["cart"] = [
  {
    "id": 1,
    "name": "Product Name",
    "price": 100,
    "quantity": 2,
    "category": "electronics",
    "customProp": "custom value"
  },
  {
    "id": 2,
    "name": "Another Product",
    "price": 50,
    "quantity": 1,
    "category": "books"
  }
]
```

---

# Core API

## Cart Class

The `Cart` class is the core of the system. It manages cart state and operations.

### Constructor

```
constructor(options?: {
  storage?: "localStorage" | "sessionStorage";
  key?: string;
})
```

**Parameters:**

- `storage` (optional): Storage backend, defaults to `"localStorage"`
- `key` (optional): Storage key identifier, defaults to `"cart"`

**Example:**

```
const cart = new Cart({ storage: "localStorage", key: "my_cart" });
```

### Methods

#### addItem(item)

Adds or increments an item in the cart.

**Signature:**

```
addItem(item: CartItem): void
```

**Behavior:**

- If an item with the same `id` exists, increments its quantity
- If new, adds it with quantity 1 (or specified quantity)
- Persists to storage immediately

**Example:**

```
cart.addItem({
    id: 1,
    name: "Laptop",
    price: 50000,
```

```
    quantity: 1,
    category: "electronics",
});
```

---

#### removeItem(id)

Removes an item from the cart by its ID.

**Signature:**

```
removeItem(id: string | number): void
```

**Behavior:**

- Completely removes the item from the cart
- No quantity decrement; item is deleted entirely
- Persists to storage immediately

**Example:**

```
cart.removeItem(1);
```

---

#### clear()

Empties the entire cart.

**Signature:**

```
clear(): void
```

**Behavior:**

- Removes all items from the cart
- Persists empty array to storage

**Example:**

```
cart.clear();
```

---

#### getItems(category?)

Retrieves items from the cart with optional category filtering.

**Signature:**

```
getItems(category?: string): CartItem[]
```

**Parameters:**

- `category` (optional): If provided, returns only items matching this category

**Returns:** Array of `CartItem` objects

**Example:**

```
const allItems = cart.getItems();
const electronics = cart.getItems("electronics");
```

---

#### getTotal(category?)

Calculates the total price of items in the cart.

**Signature:**

```
getTotal(category?: string): number
```

**Parameters:**

- `category` (optional): If provided, calculates total for only items in this category

**Returns:** Total price as a number (sum of `price × quantity` for all items)

**Calculation:**

```
Total = Σ(item.price × item.quantity) for all matching items
```

**Example:**

```
const totalPrice = cart.getTotal();
const electronicsCost = cart.getTotal("electronics");
```

---

## Storage Layer

### getStorage(type)

Retrieves the appropriate storage object.

**Signature:**

```
function getStorage(type: "localStorage" | "sessionStorage"): Storage;
```

**Returns:** Window's `localStorage` or `sessionStorage` object

**Throws:** Error if `window` object is not available (server-side)

**Internal Use:** This function is used internally by the `Cart` class and is typically not called directly by users.

---

# React Integration

## CartProvider Component

Context provider that wraps your React application to enable cart functionality.

**Signature:**

```
function CartProvider(props: {
    children: React.ReactNode;
    storage?: "localStorage" | "sessionStorage";
    key?: string;
}): JSX.Element;
```

**Props:**

- `children`: React components to be wrapped
- `storage`: Storage backend (default: `"localStorage"`)
- `key`: Storage key identifier (default: `"cart"`)

**Example:**

```
import { CartProvider } from "@ravishranjan/cart/react";

export default function App() {
  return (
    <CartProvider storage="localStorage" key="my_store">
      <YourAppComponents />
    </CartProvider>
  );
}
```

---

## useCart Hook

React hook for accessing cart functionality within a component.

**Signature:**

```
function useCart(): CartContextType;
```

**Returns:** Cart context object with methods and state

**Throws:** Error if used outside of `CartProvider`

**Context Type:**

```
type CartContextType = {
    cart: CartItem[]; // Current items in cart
    addItem: (item: CartItem) => void;
    removeItem: (id: string | number) => void;
    clear: () => void;
    total: number; // Current cart total
};
```

**Example:**

```
import { useCart } from "@ravishranjan/cart/react";

function ShoppingCart() {
  const { cart, addItem, removeItem, clear, total } = useCart();

  return (
    <div>
      <ul>
        {cart.map(item => (
          <li key={item.id}>
            {item.name} - ₹{item.price}
            <button onClick={() => removeItem(item.id)}>Remove</button>
          </li>
        ))}
      </ul>
      <p>Total: ₹{total}</p>
      <button onClick={clear}>Clear Cart</button>
    </div>
  );
}
```

---

## Sync Behavior

The `CartProvider` automatically synchronizes cart state across browser tabs/windows using the `storage` event.

**How it Works:**

1. `CartProvider` attaches a listener to the `storage` event
2. When cart is modified in one tab, the event fires in all other tabs
3. Cart state updates automatically in listening components
4. No manual refresh or polling required

**Use Case:** Open your store in multiple tabs; adding items in one tab reflects in all others.

---

# Storage Management

## Persistence Mechanism

Cart items are stored as a JSON string in the browser's storage.

**Storage Format:**

```
{
    "cart": "[{\"id\":1,\"name\":\"Item\",\"price\":100,\"quantity\":2}]"
}
```

## Storage Options

| Option | Scope | Persistence | Use Case |
|---|---|---|---|
| **localStorage** | Same origin | Persists across browser sessions | Default; for e-commerce carts |
| **sessionStorage** | Same tab | Clears when tab is closed | Temporary carts; demo purposes |

## Storage Key

By default, items are stored under the key `"cart"`. This can be customized:

```
// Native Cart
const cart = new Cart({ key: "user_123_cart" });

// React Provider
<CartProvider key="user_123_cart">
```

Multiple cart instances can coexist in the same storage if using different keys.

---

# Type Definitions

## CartItem

Core interface for items in the cart.

```
interface CartItem {
    id: string | number; // Unique identifier (required)
    name: string; // Item name (required)
    price?: number; // Price per unit (optional)
    quantity?: number; // Quantity in cart (optional, defaults to 1)
    category?: string; // Category identifier (optional)
    [key: string]: any; // Custom properties allowed
}
```

**Field Details:**

| Field | Type | Required | Notes |
|---|---|---|---|
| `id` | string \| number | ✓ | Used to identify and deduplicate items |
| `name` | string | ✓ | Display name for the item |
| `price` | number | ✗ | Price per unit; defaults to 0 if omitted |
| `quantity` | number | ✗ | Number of units; defaults to 1 if omitted |
| `category` | string | ✗ | Used for filtering via `getItems(category)` |
| Custom props | any | ✗ | Additional properties are preserved |

**Example:**

```
const item: CartItem = {
    id: 101,
    name: "Wireless Headphones",
    price: 2499,
    quantity: 1,
    category: "electronics",
    color: "black", // Custom property
    manufacturer: "Sony", // Custom property
};
```

## CartContextType

Context API type for React integration.

```
type CartContextType = {
    cart: CartItem[];
    addItem: (item: CartItem) => void;
    removeItem: (id: string | number) => void;
    clear: () => void;
    total: number;
};
```

---

# Configuration

## Build Configuration

The project uses TypeScript with the following tooling:

**Build Tools:**

- **tsup**: Bundles TypeScript into multiple formats (ESM, CJS)
- **esbuild**: Creates minified global/UMD bundle for CDN
- **TypeScript**: Compiles and generates type definitions

**Build Scripts:**

```
{
    "build": "tsup src --format esm,cjs --dts",
    "build:cdn": "esbuild src/core/cart.global.ts --bundle --minify --
format=iife --global-name=Cart --platform=browser --
outfile=dist/cart.min.js",
    "dev": "tsup src --watch",
    "prepare": "npm run build && npm run build:cdn"
}
```

## Export Configuration

The package exports multiple entry points for different use cases:

```
{
    "exports": {
        ".": {
            "types": "./dist/index.d.ts",
            "import": "./dist/index.mjs",
            "require": "./dist/index.js"
        },
        "./react": {
            "types": "./dist/react/index.d.ts",
            "import": "./dist/react/index.mjs",
            "require": "./dist/react/index.js"
        }
    }
}
```

**Import Paths:**

```
// Core Cart API
import Cart from "@ravishranjan/cart";
import { Cart } from "@ravishranjan/cart";
```

```
// React bindings
import { CartProvider, useCart } from "@ravishranjan/cart/react";
```

---

# Build & Distribution

## NPM Registry

- **Package Name:** `@ravishranjan/cart`
- **Registry:** https://npmjs.com/package/@ravishranjan/cart
- **Scope:** `@ravishranjan`

## CDN Distribution

The package is distributed on multiple CDNs for easy script tag inclusion:

| CDN | URL |
|---|---|
| **jsDelivr** | `https://cdn.jsdelivr.net/npm/@ravishranjan/cart/dist/cart.min.js` |
| **unpkg** | `https://unpkg.com/@ravishranjan/cart/dist/cart.min.js` |

**Global Variable Name:** `Cart` (when included via CDN)

## Build Process

1. **Development:** Run `npm run dev` to watch and rebuild on changes
2. **Production:** Run `npm run build` to generate optimized output
3. **Publish:** Run `npm publish` (prepare script runs automatically)

## Pre-publish Hook

The `prepare` script runs automatically before `npm publish`:

```
npm run build && npm run build:cdn
```

This ensures all distributions are up-to-date before publishing to npm.

---

# Installation & Setup

## From NPM

```
npm install @ravishranjan/cart
```

## From CDN

```
<script
src="https://cdn.jsdelivr.net/npm/@ravishranjan/cart/dist/cart.min.js">
</script>
<script>
    const cart = new Cart();
</script>
```

## Peer Dependencies

React integration requires React 18+:

```
{
    "peerDependencies": {
        "react": ">=18.0.0 <20.0.0",
        "react-dom": ">=18.0.0 <20.0.0"
    }
}
```

Note: React is optional for the core cart API.

---

# Error Handling

## Common Errors

| Error | Cause | Solution |
| --- | --- | --- |
| `useCart must be used within CartProvider` | Hook called outside provider | Wrap components with `<CartProvider>` |
| `No window object found` | Storage function called server-side | Use dynamic imports or check for `typeof window` |

## Performance Considerations

1. **Storage Access:** Each operation reads/writes to browser storage. Minimize frequent operations.
2. **Large Carts:** With many items, `getTotal()` recalculates on every call. Cache if needed.
3. **Sync Events:** Large carts may cause lag when syncing across multiple tabs.

---

## Browser Compatibility

- **localStorage/sessionStorage:** IE8+, all modern browsers
- **React 18+:** All modern browsers
- **TypeScript types:** Requires TypeScript 4.5+

---

## License